Міністерство освіти і науки України

Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського»

Комп’ютерного практикуму №3
з дисципліни «Криптографія»

Тема: «Криптоаналіз афінної біграмної підстановки»

Виконали: Валюх, Журавський

Група: ФБ-43

Варіант: №3

Київ — 2026

Тема та мета роботи

Тема: Криптоаналіз афінної біграмної підстановки.

Мета: Набуття навичок частотного аналізу на прикладі розкриття моноалфавітної підстановки; опанування прийомами роботи в модулярній арифметиці.

Постановка задачі

- Реалізувати математичні підпрограми: розширений алгоритм Евкліда, обчислення оберненого елемента за модулем і розв'язування лінійних порівнянь з урахуванням усіх можливих розв'язків.
- Знайти п'ять найчастіших біграм шифротексту свого варіанта.
- Перебрати можливі зіставлення частих біграм мови й шифротексту та сформувати кандидати на ключ (a, b).
- Розшифрувати текст кандидатними ключами та автоматично відкинути неосмислені результати.
- Повторювати пошук і перевірку до отримання осмисленого тексту.

Вихідні дані та програмне середовище

Варіант №3. Шифротекст: файл «03 (1).txt» (на Kali Linux під час виконання використовувався файл з назвою «03 (1)(1).txt»). Мова відкритого тексту — російська, без пробілів і знаків пунктуації. Використано Python 3 у Kali Linux.

Алфавіт: абвгдежзийклмнопрстуфхцчшщьыэюя; m = 31; модуль $m^2 = 961$. Символам присвоюються номери від 0 до 30.

Для біграми (x_1, x_2) використано $X = 31x_1 + x_2$. Шифрування: $Y = (aX + b) \bmod 961$. Розшифрування: $X = a^{-1}(Y - b) \bmod 961$.

Хід роботи

1. Математичні операції та підготовка даних

Відповідно до методички реалізовано функції egcd(), inverse(), solve_linear(). Вони фактично містяться у файлі step3_find_keys.py. Функція egcd реалізує розширений алгоритм Евкліда; inverse повертає обернений елемент, якщо він існує; solve_linear повертає всі розв'язки порівняння або порожній список.

Окремий скрипт step1_prepare.py читає файл, усуває символи поза алфавітом, переводить літери до нижнього регістру та записує очищений шифротекст до step1_clean.txt. Відбувається розбиття на біграми без перекриття.

Показник	Значення
Модуль	961
Символів у шифротексті	5630
Біграм без перекриття	2815
Файл результату	step1_clean.txt

Приклад запуску на Kali Linux:

```
python3 step1_prepare.py "03 (1)(1).txt"
```

2. Частотний аналіз біграм

За допомогою step2_frequencies.py та Counter обчислено кількості біграм, узятих на позиціях (1,2), (3,4) і т. д. Відносну частоту обчислено як кількість появ / 2815.

Біграма	Кількість	Частота

Біграма	Кількість	Частота
тд	77	2,74 %
рб	53	1,88 %
во	52	1,85 %
щю	45	1,60 %
кд	42	1,49 %

П’ять найчастіших біграм російської мови, прийнятих за методичкою: «ст», «но», «то», «на», «ен».

```
python3 step2_frequencies.py
```

3. Формування можливих ключів

Програмою step3_find_keys.py виконано зіставлення впорядкованих пар із п’яти частих біграм мови та шифротексту. Для кожного зіставлення розв’язано систему:

$Y_1 \equiv aX_1 + b \pmod{961}$; $Y_2 \equiv aX_2 + b \pmod{961}$.

Після віднімання: $a(X_1 - X_2) \equiv Y_1 - Y_2 \pmod{961}$. Для кожного допустимого a обчислювалося $b \equiv Y_1 - aX_1 \pmod{961}$. Значення a без оберненого елемента відкидалися.

Функція solve_linear коректно враховує випадки gcd(A,961)>1: якщо права частина не ділиться на gcd, розв’язків немає; інакше повертаються всі допустимі розв’язки.

Показник	Результат
Варіантів зіставлення впорядкованих пар	400
Допустимих розв’язків у циклі перебору (з повтореннями)	448
Різних ключів (a, b)	218
Ключів після статистичної перевірки	1

Важливо: кількість 448 — лічильник допустимих розв’язків у програмі, який може включати повтори; 218 — число унікальних ключів.

```
python3 step3_find_keys.py
```

4–5. Автоматичний відбір і дешифрування

У step3_find_keys.py кожен новий кандидат на ключ застосовується до повного шифротексту, після чого перевіряється функцією meaningful(). Вона обчислює три статистичні ознаки:

f = (No + Na + Ne) / N — сумарна відносна частота частих літер;

r = (Nф + Nщ + Nь) / N — сумарна відносна частота рідкісних літер;

bad = кількість заборонених сусідніх пар / (N − 1). Для третьої ознаки враховуються всі сусідні пари, включно з тими, що перетинають межі шифрованих біграм.

Кандидат визнається перспективним, якщо f > 0,22; r < 0,06; bad < 0,005. Ці пороги є обраними параметрами власного програмного розпізнавача, а не універсальним доведенням осмисленості.

Параметр	Фактичне значення	Умова
Часті літери (о, а, е)	0,276021	> 0,22
Рідкісні літери (ф, щ, ь)	0,025933	< 0,06
Заборонені біграми	0,000711	< 0,005

Після автоматичного відбору залишився один ключ: a = 199, b = 700. Скрипт step4_decrypt.py зчитує цей ключ зі step3_keys.json і записує фінальний текст до step4_decrypted.txt.

Обернений елемент $a^{-1} = 198$, оскільки $199 \cdot 198 = 39402 \equiv 1 \pmod{961}$. Отже, $X = 198(Y - 700) \bmod 961$.

```
python3 step4_decrypt.py
```

Початок відновленого тексту (як у фактичному файлі, без виправлення орфографії):

```
отцеубийствокакизвестноосновноеиизначальноепреступлениечеловечестваиотдельногочеловекав
овсякомслучаеоноглавныйисточникчувствавинынеизвестноединственныйлиисследованиямнеуда
лосьещеустановитьдушевноепроисхождениевиныипотребностиискупленияноотнюдьнесуществен
ноединственныйлиэтоисточникпсихологическоеположениесложноинуждаетсявобъясненияхотнош
ениемальчикак…
```

Розшифрування дало зв’язний текст російською мовою, що разом зі статистичним відбором підтверджує успішне знаходження ключа.

Проблеми, які виникли, та їх усунення

Під час першого запуску виникла помилка FileNotFoundError через відмінність назви шифротексту «03 (1).txt» від реальної назви файлу в Kali Linux «03 (1)(1).txt». Після запуску з правильною назвою перший етап успішно створив step1_clean.txt; наступні скрипти змогли використати його й завершили роботу.

Перевірка отриманого результату

1. Установлено, що gcd(199, 961) = 1 та 199·198 mod 961 = 1. 2. Після застосування алгоритму розшифрування отримано зв’язний текст. 3. Для додаткової технічної перевірки можна повторно зашифрувати step4_decrypted.txt ключем (199,700) і порівняти з step1_clean.txt. Така перевірка підтверджує оборотність перетворення для ключа, а змістовність підтверджує правильність вибору ключа.

Відповіді на контрольні питання

1. Які статистичні властивості використано?

Частоти неперекривних біграм шифротексту зіставлялися з типовими найчастішими біграмами російської мови. Афінна біграмна підстановка переставляє біграми, зберігаючи кількість появ кожної з них.

2. Чому не кожне зіставлення приводить до допустимого ключа?

Лінійне порівняння може не мати розв’язків. Навіть за наявності розв’язку параметр a мусить бути взаємно простим із 961, щоб існував обернений елемент. Частотні відповідності також можуть бути хибними.

3. Чи могли виникати кілька кандидатів з одного зіставлення?

Так. Якщо $\gcd(X_1 - X_2, 961) > 1$ і різниця $Y_1 - Y_2$ ділиться на цей дільник, лінійне порівняння має кілька розв’язків. solve_linear() повертає їх усі; непридатні a відкидаються, а повторні ключі прибираються множиною possible.

4. Як визначено правильний ключ?

Серед 218 різних ключів єдиний кандидат (199,700) задовольнив статистичні пороги розпізнавача. Після остаточного дешифрування отримано осмислений російськомовний текст.

5. Який критерій використано і чому?

Комбінований критерій: сумарна частота частих літер «о», «а», «е», сумарна частота рідкісних «ф», «щ», «ь» та частота заборонених біграм. Його легко автоматизувати та він допомагає відсіяти випадкові тексти.

6. Як поводився критерій для правильного та неправильних ключів?

Правильний ключ дав f≈0,2760, r≈0,0259, bad≈0,00071 і пройшов усі пороги. Решта 217 унікальних кандидатів не виконали принаймні одну з умов та були відкинуті.

Висновки

Під час комп’ютерного практикуму №3 виконано криптоаналіз афінної біграмної підстановки для варіанта №3. За допомогою чотирьох скриптів Python здійснено підготовку шифротексту, частотний аналіз, пошук ключів шляхом розв’язування лінійних порівнянь і автоматичне розпізнавання осмисленого тексту. З 218 різних ключів статистичну перевірку пройшов один: (199,700). Отримано зв’язний розшифрований текст довжиною 5630 символів. Мету практикуму досягнуто.